

MPC-PiNNs technique applied to Quadrotors

Authors

January 22, 2025

Todo list

Spiega modello quadrotor	7
Aggiungere figura MLP	7
definition of losses	9
image of pinn time	10
pipeline (prendi foto paper manipulator)	10
what we have obtained	10
problems	10
pinn vs pinnc	10

Contents

Todo list	1
1 Introduction	3
2 Models	3
2.1 Unicycle kinematic model	3
2.2 Quadrotor dynamic model	4
3 Model Predictive Control	4
3.1 Cost function	5
3.2 Constraints	6
3.3 Unicycle	6
3.4 Quadrotor	7
4 Dataset	7

5	Simple Neural Network	7
5.1	Unicycle Neural model	7
5.1.1	Architecture	7
5.1.2	Dataset	8
5.1.3	Training	8
5.2	Quadrotor Neural model	9
6	Physical Informed Neural Network	9
7	Results	10
8	Conclusions	10

1 Introduction

Model Predictive Control (MPC) has emerged as a powerful control strategy in robotics thanks to its ability to handle constraints and optimize system behavior over a prediction horizon. When combined with neural networks, which can learn complex system dynamics, it creates a promising framework for controlling nonlinear systems like UAVs.

This report presents the implementation and analysis of a Neural Model Predictive Control (NMPC) scheme applied to a quadrotor. We followed a bottom-up approach, testing the pipeline first on a Unicycle, which still has a nonlinear dynamics, but is easier to treat due a simpler model and faster prototyping possibilities. Our approach integrates Physical Informed Neural Network to approximate the quadrotor’s dynamics, which are then utilized within an MPC framework to generate optimal control inputs. We explore the effectiveness of this method in trajectory tracking tasks, comparing it with traditional control approaches.

The following sections detail the theoretical foundations, implementation specifics, and experimental results of our neural MPC system. We also discuss the challenges encountered and potential improvements for future work.

2 Models

Our scheme uses the MPC to generate optimal control inputs while the Neural part is used to model the dynamics and kinematics of our system. The Unicycle has been modeled with its kinematic model while for the quadrotor we have used its dynamic model.

2.1 Unicycle kinematic model

The kinematic model of the unicycle is a simplified representation of the movement of the vehicle in terms of position and orientation. This simple model has been derived directly from the so-called "rolling without slipping constraint". The state space is represented by the robot cartesian position and orientation $[x, y, \theta]$, while the input actions are represented by the driving and steering velocities the vehicle can assume $[v, \omega]$. The kinematic model of a unicycle robot it is described by the following equations:

$$\begin{cases} \dot{x}(t) = v(t) \cos(\theta(t)) \\ \dot{y}(t) = v(t) \sin(\theta(t)) \\ \dot{\theta}(t) = \omega(t) \end{cases} \quad (1)$$

2.2 Quadrotor dynamic model

For the quadrotor, we have used its dynamic model. We can describe this second order model by starting from the two following equations:

$$\begin{cases} m\ddot{p} = f_t + f_g + f_a + f_d \\ J\dot{\omega} + \omega \times (J\omega) = \tau \end{cases} \quad (2)$$

where the first equation, which is derived from Newton's Second Law, represents the Translational Dynamics while the second one, derived from Euler's equations, represents the Rotational Dynamics. In the first equation f_t represents the total thrust force: $f_t = R \cdot [0 \ 0 \ T]^T$, where R is the rotation matrix of (ϕ, θ, ψ) , the roll, pitch and yaw angles. The total thrust force T produced by the rotors is defined by the following equation:

$$T = f_1 + f_2 + f_3 + f_4, \quad \text{with} \quad f_i = k_f \omega_i^2 \quad (3)$$

where k_f is the thrust coefficient and ω_i is the angular speed of the i -th rotor. For the Rotational Dynamics, τ represents the torques in the body frame and can be expressed as follows:

$$\tau = \begin{bmatrix} \tau_\phi \\ \tau_\theta \\ \tau_\psi \end{bmatrix} = \begin{bmatrix} L(f_1 - f_3) \\ L(f_2 - f_4) \\ \tau_\psi \end{bmatrix} \quad (4)$$

where L is the distance from the center of mass of each rotor.

Given the previous equations, if we assume aereodynamics and gyroscopic effects to be negligible and that we have small roll and pitch angles, we can derive the simplified model fo the quadrotor:

$$\begin{cases} \ddot{x} = -(\cos \psi \sin \theta \cos \phi + \sin \psi \sin \phi) \frac{T}{m} \\ \ddot{y} = -(\sin \psi \sin \theta \cos \phi - \cos \psi \sin \phi) \frac{T}{m} \\ \ddot{z} = -(\cos \theta \cos \phi) \frac{T}{m} + g \\ \ddot{\phi} = \frac{\tau_\phi}{I_x} \\ \ddot{\theta} = \frac{\tau_\theta}{I_y} \\ \ddot{\psi} = \frac{\tau_\psi}{I_z} \end{cases} \quad (5)$$

3 Model Predictive Control

Model Predictive Control (MPC), it is an optimal control technique where the actions are chosen solving an optimization problem, composed by a set of constraints

and an objective function to be minimized over a finite horizon.

$$\min_{u(0), u(1), \dots, u(N-1)} \sum_{k=0}^{N-1} \ell(x(k), u(k)) + \ell_f(x(N)) \quad (6)$$

$$\text{s.t. } x(k+1) = f(x(k), u(k)), \quad k = 0, \dots, N-1, \quad (7)$$

$$x(k) \in \mathcal{X}, \quad u(k) \in \mathcal{U}, \quad k = 0, \dots, N-1, \quad (8)$$

$$x(0) = x_0 \quad (9)$$

Here, $\mathcal{U}$ and $\mathcal{X}$ denote the admissible sets for inputs and states, respectively, while x_0 is the initial state. One of the main advantages of this control approach is the ability to handle multivariable interactions and constraints. However, solving the above optimization problem in real time can be computationally challenging, especially for large-scale or highly nonlinear systems. This is one reason why neural networks have been employed to either approximate the system model or enhance the optimization process. In our case, we have used Neural Networks to predict the next state after the application of the control input given by the MPC.

3.1 Cost function

Our objective is to minimize at each step the error with respect to the reference trajectory, both in traslational and orientational dimension, while also maintaining the module of the input actions as low as possible. A terminal state term is present too, where the traslational and orientational errors of the last horizon step are tried to be minimized. Thus, the cost function is composed by the following three terms:

1. **Spatial error:** at each step of the horizon, the squared errors over the state variables are summed together

$$J_{sp_k} = W_x \sum_{i=0}^n x_{i_k}^2 \quad (10)$$

where x_{i_k} it is the i -th component of the state vector at the k -th instant.

2. **Terminal state:** to make sure the final state over the horizon is good, we add a related penalty term in the terminal state N :

$$J_{sp_N} = W_x \sum_{i=0}^n x_{i_N}^2 \quad (11)$$

where x_{i_k} it is the i -th component of the state vector at the last instant over the horizon

3. **Input:** it penalizes high values of input controls

$$J_{u_k} = W_u \sum_{i=0}^n u_{i_k}^2 \quad (12)$$

where u_{i_k} it is the i -th component of the control vector at the k -th instant.

3.2 Constraints

The constraints of the NLP concerns some limitations on the state of the vehicle model and on the dynamics of the system. In detail, we have just two constraints:

1. **Initial state:** the initial state of the unicycle model must be the current measured state. Therefore, defining the vehicle state as x :

$$x_0 = x_{current} \quad (13)$$

2. **Dynamics:** given the robot state at a certain time-step as x_t , an input at the same time-step as u_t and the dynamic model of the system defined as f_{expl} , we establish that the next state is obtained by applying the input at the given state according to dynamic model:

$$x_{t+1} = f_{expl}(x_t, u_t) \quad (14)$$

3.3 Unicycle

For our formulation, we have considered a discrete-time version of the model equations (1) obtained via Euler integration:

$$\begin{cases} x_{k+1} = x_k + T_s v_k \cos \theta_k, \\ y_{k+1} = y_k + T_s v_k \sin \theta_k, \\ \theta_{k+1} = \theta_k + T_s \omega_k. \end{cases} \quad (15)$$

According to this, the cost function terms are the following ones:

1. **Spatial error:** at each step of the horizon, the squared errors along x-axis, y-axis and error of orientation are summed together

$$J_{sp} = W_{e_x} e_x^2 + W_{e_y} e_y^2 + W_{e_\theta} e_\theta^2 \quad (16)$$

where

$$e_x = x - x_{ref} \quad (17a)$$

$$e_y = y - y_{ref} \quad (17b)$$

$$e_\theta = \theta - \theta_{ref} \quad (17c)$$

2. **Terminal state:** to make sure the final stage is a good one, we add a penalty on errors along x-axis, y-axis and error of orientation in the terminal state

$$J_{term} = W_{e_{x_N}} e_{x_N}^2 + W_{e_{y_N}} e_{y_N}^2 + W_{e_{\theta_N}} e_{\theta_N}^2 \quad (18)$$

3. **Input:** it penalizes high values of input controls

$$J_u = W_v v^2 + W_\omega \omega^2 \quad (19)$$

3.4 Quadrotor

Spiega modello quadrotor

4 Dataset

5 Simple Neural Network

The model must be well-defined to be used in NMPC, in such a way to obtain reliable predictions of system behavior in response to various control actions. The usage of a data-driven approach, such as neural networks, in the vehicle modeling phase has the advantage of not modeling all the nonlinear physical effects, which can be also very complex. In fact, neural networks have the capability to accurately model them. Neural networks are universal approximators, and they can be used both for designing the entire dynamic model of a system and approximating the disturbances and noises a system can be affected. Our purpose is to approximate the entire kinematic model of a quadrotor with a neural model, by first testing the performances on a unicycle model due to its low complexity. In this section we will start describing briefly the kinematic model of a unicycle, and then we will describe the neural architecture used to approximate it.

5.1 Unicycle Neural model

Aggiungere figura MLP

5.1.1 Architecture

The neural network used to model the kinematic model of the unicycle is a simple MLP architecture, with two hidden layers of 128 neurons. The network has two input which are concatenated together, the current state of the unicycle (dimension

3) and the current applied actions on the vehicle (dimension 2), so it must be of dimension 5 and is represented by

$$X = [x, y, \theta, v, \omega] \quad (20)$$

The output of the network has dimension 3, and it represents the dynamics of the system, therefore it reproduces the differential equations which describe the motion of the unicycle:

$$Y = [\dot{x}, \dot{y}, \dot{\theta}] \quad (21)$$

5.1.2 Dataset

The dataset was generated by using the nominal model of the robot and generating 500 trajectories. Each trajectory is composed by 1000 steps, where the inputs are maintained constant for the whole episode. This choice was taken because it was noticed that the interval between consecutive time-steps is too small, therefore the changes about the state of the unicycle are small too, and the network was not able to approximate correctly the motion of the vehicle. On the contrary, having the same input for a lot of steps allows to understand in a better way the dynamics of the system subject to specific inputs.

Each sample is not just a pair state-action at step t , but it is a window of dimension k which contains all pairs state-action from time-step t till $t+k$ (less the last action a_{t+k} which is useless), so it is in the form

$$(s_t, a_t, s_{t+1}, a_{t+1}, \dots, s_{t+k-1}, a_{t+k-1}, s_{t+k}) \quad (22)$$

This allows to have a larger window of evolution of the system with respect to just the motion executed in one single step.

5.1.3 Training

As can be noticed in the previous section, the dataset does not contain the dynamics of the system, but just the evolution of the states. On the contrary, the forward pass of the network returns the vehicle dynamics. So, what we did is to pass the first pair (s_t, a_t) of a sample to the network; then we integrated the output in the sample time dt , and added to the previous state (which is also the input of the network). In this way we obtain the state s_{t+1} . At this point we concatenate the new obtained state with the action a_{t+1} given by the sample dataset and we pass it to the network again, and repeating this process k times we get the states evolution according to the neural model.

Ended this mechanism, the loss is computed as the mean-squared error between the neural and the nominal evolutions.

5.2 Quadrotor Neural model

6 Physical Informed Neural Network

Traditional methods for solving differential equations, such as finite element or finite difference methods, often require discretization of the domain and can be computationally expensive for high-dimensional problems. Physical Informed Neural Networks (PiNNs) propose an alternative approach where the solution is approximated by a neural network that is trained not only on data but also by enforcing the underlying physical laws via the differential equations. The overall loss function $\mathcal{L}$ used is composed by the sum of two terms, which are the data loss $\mathcal{L}_{data}$ and the physics loss $\mathcal{L}_{phy}$.

$$\mathcal{L}(\theta) = \mathcal{L}_{data}(\theta) + \mathcal{L}_{phys}(\theta), \quad (23)$$

The first term defines the discrepancy between the prediction of the network and the observed data, while the second one specifies the correct modeling of the differential equation. Usually $\mathcal{L}_{data}$ is define as the mean squared error between the prediction and the ground truth, while $\mathcal{L}_{phy}$ is defined using the residual of the differential equation.

$$\mathcal{L}_{data}(\theta) = \frac{1}{N_u} \sum_{i=1}^{N_u} |u_{\theta}(\mathbf{x}_i, t_i) - u_i^{obs}|^2 \quad (24)$$

$$\mathcal{L}_{phys}(\theta) = \frac{1}{N_r} \sum_{j=1}^{N_r} |\mathcal{N}[u_{\theta}](\mathbf{x}_j, t_j)|^2, \quad (25)$$

As previously stated, we have used a bottom-up approach by first testing this approach to a Unicycle. This robot is characterized by simple kinematic equations. Its configuration vector is composed by the cartesian position (x, y) and its orientation angle θ , while the control inputs are the driving velocity v and the steering velocity w . We have used the PiNN to predict the next state q_{k+1} of the robot given the current state, the control input given by the MPC and the time instant. Over a small time instant we have chosen to use Euler integration to do the approximation of the next state: $q_{k+1} = q_k + \Delta t \cdot f(q_k, u_k)$, where the dynamic function is defined as follows.

$$f(q_k, u_k) = \begin{cases} x_{k+1} = v_k \cos(\theta_k) \\ y_{k+1} = v_k \sin(\theta_k) \\ \theta_{k+1} = w_k \end{cases} \quad (26)$$

definition of losses

image of pinn time

pipeline (prendi foto paper manipulator)

7 Results

what we have obtained

problems

pinn vs pinnc

8 Conclusions

References